In [12]:
```
import os
print(os.getcwd())
```

```
/home/Eexam5/Documents
```

In [13]:
```
!pip install pandas
```

```
Requirement already satisfied: pandas in /home/ssn-29/anaconda3/lib/python3.13/site-packages (2.3.3)
Requirement already satisfied: numpy>=1.26.0 in /home/ssn-29/anaconda3/lib/python3.13/site-packages (from pandas)
(2.3.5)
Requirement already satisfied: python-dateutil>=2.8.2 in /home/ssn-29/anaconda3/lib/python3.13/site-packages (from p
andas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /home/ssn-29/anaconda3/lib/python3.13/site-packages (from pandas) (20
24.1)
Requirement already satisfied: tzdata>=2022.7 in /home/ssn-29/anaconda3/lib/python3.13/site-packages (from pandas)
(2025.2)
Requirement already satisfied: six>=1.5 in /home/ssn-29/anaconda3/lib/python3.13/site-packages (from python-dateutil
>=2.8.2->pandas) (1.17.0)
```

In [17]:
```
print(df.columns)
```

```
Index(['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
       'petal width (cm)', 'target'],
      dtype='object')
```

In [23]:
```
import pandas as pd
columns=["sepal length(cm)","sepal width(cm)","petal length(cm)","petal width(cm)","target"]
#LOADING THE DATASET
df=pd.read_csv("iris.data",header=None,names=columns)
df.head();
```

In [24]:
```
#NO OF FEATURES AND SIZE
print(df.head)
print(df.columns)
print(df.shape)
```

```
<bound method NDFrame.head of      sepal length(cm)  sepal width(cm)  petal length(cm)  petal width(cm)  \
0                 5.1               3.5               1.4               0.2
1                 4.9               3.0               1.4               0.2
2                 4.7               3.2               1.3               0.2
3                 4.6               3.1               1.5               0.2
4                 5.0               3.6               1.4               0.2
..                ...               ...               ...               ...
145               6.7               3.0               5.2               2.3
146               6.3               2.5               5.0               1.9
147               6.5               3.0               5.2               2.0
148               6.2               3.4               5.4               2.3
149               5.9               3.0               5.1               1.8

            target
0      Iris-setosa
1      Iris-setosa
2      Iris-setosa
3      Iris-setosa
4      Iris-setosa
..             ...
145  Iris-virginica
146  Iris-virginica
147  Iris-virginica
148  Iris-virginica
149  Iris-virginica

[150 rows x 5 columns]>
Index(['sepal length(cm)', 'sepal width(cm)', 'petal length(cm)',
       'petal width(cm)', 'target'],
      dtype='object')
(150, 5)
```

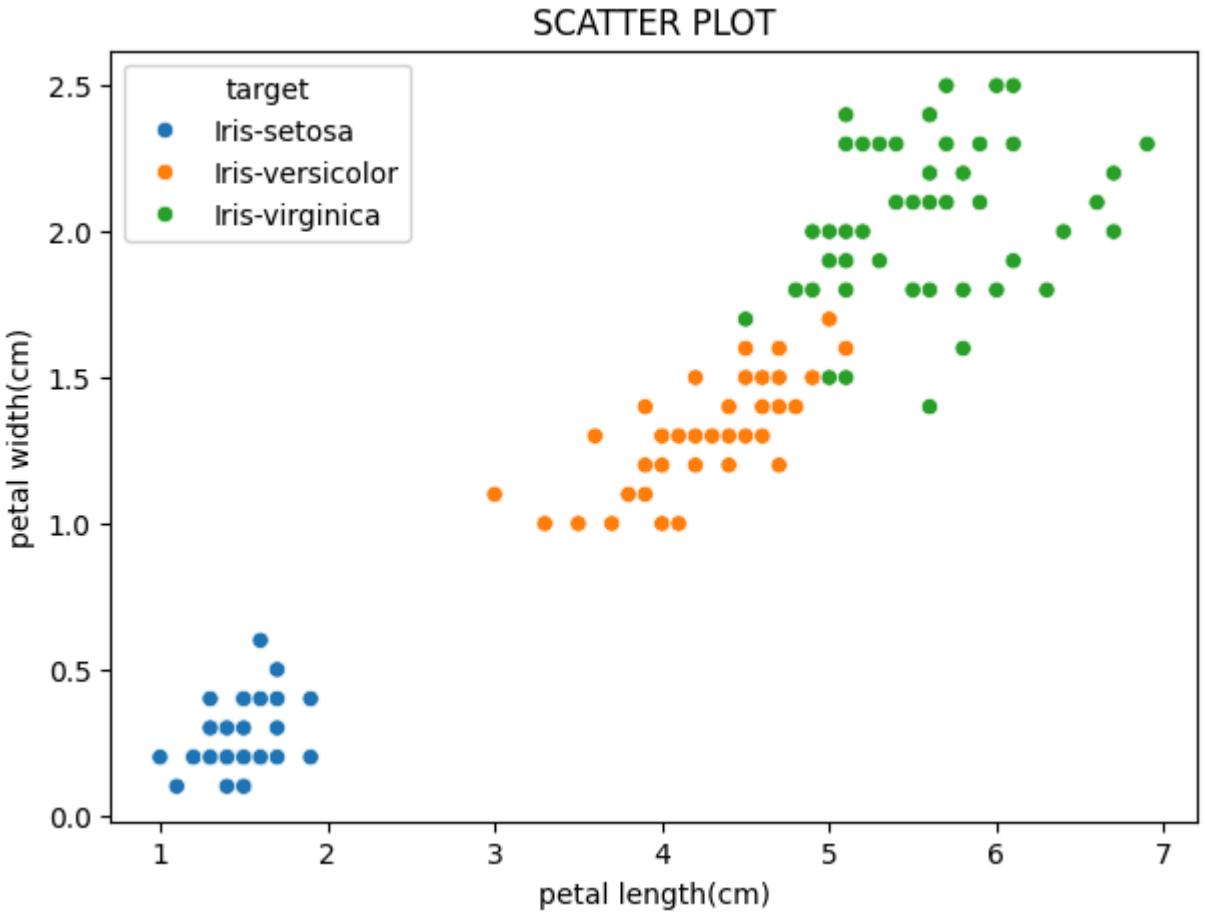
In [25]:
```
df.describe()
```

Out[25]:

	sepal length(cm)	sepal width(cm)	petal length(cm)	petal width(cm)
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

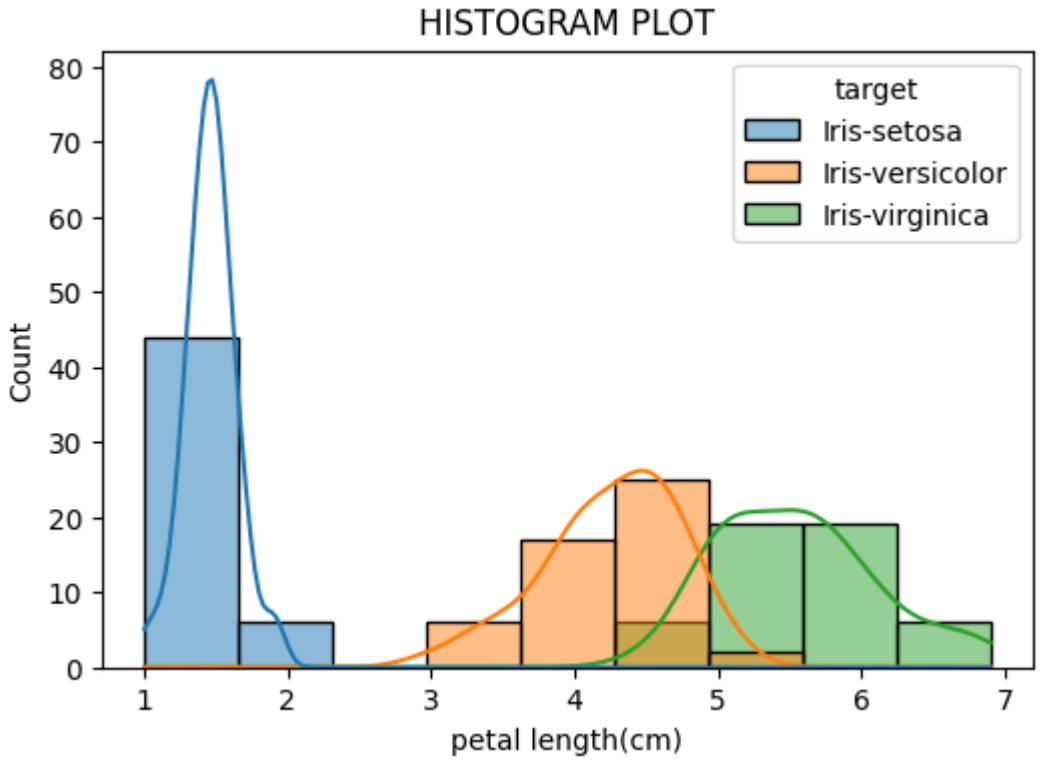
In [28]:

```
import matplotlib.pyplot as plt
import seaborn as sns
plt.figure(figsize=(7,5))
sns.scatterplot(data=df,x="petal length(cm)",y="petal width(cm)",hue="target")
plt.title("SCATTER PLOT")
plt.show()
```

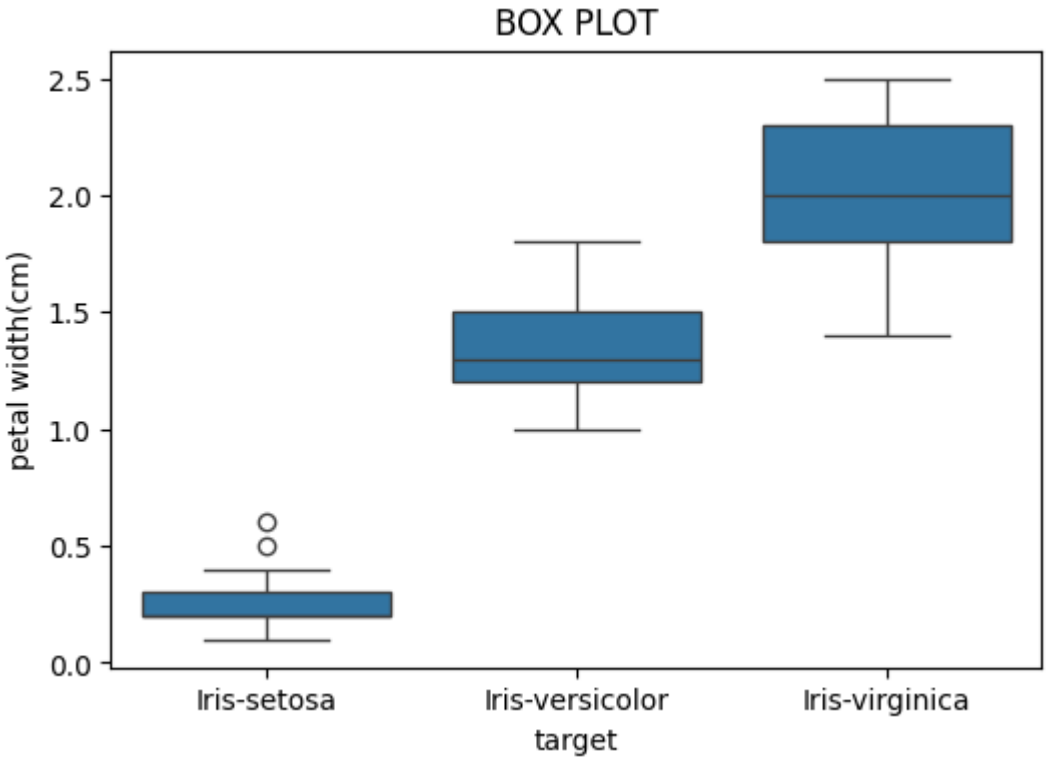


In [31]:
```
plt.figure(figsize=(6,4))
sns.histplot(data=df,x="petal length(cm)",hue="target",kde=True)
plt.title("HISTOGRAM PLOT")
plt.show()
```

```
<Figure size 600x400 with 0 Axes>
<Figure size 600x400 with 0 Axes>
```



In [32]:
```
plt.figure(figsize=(6,4))
sns.boxplot(x="target",y="petal width(cm)",data=df)
plt.title("BOX PLOT")
plt.show()
```



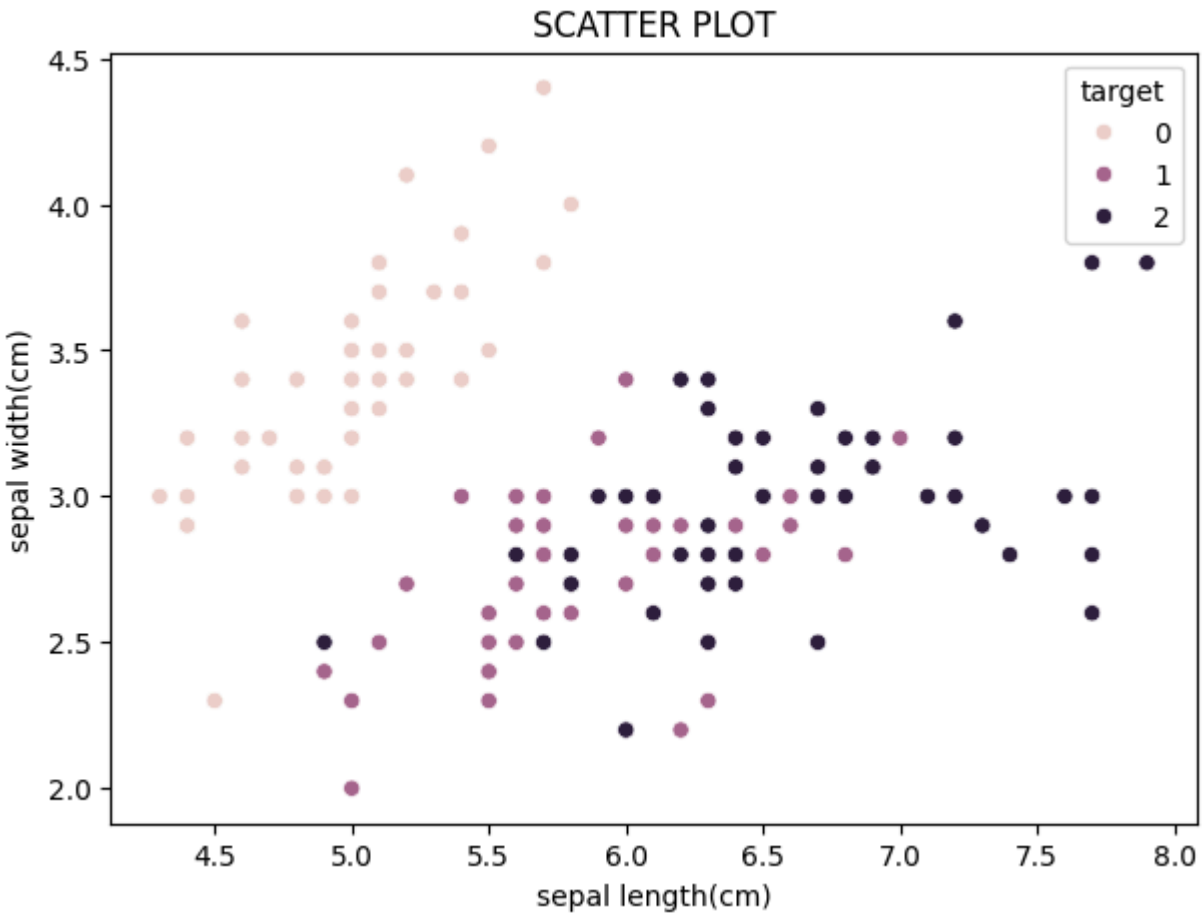
In [33]:
```
df.isnull().sum()
```

Out[33]:
```
sepal length(cm)    0
sepal width(cm)     0
petal length(cm)    0
petal width(cm)     0
target              0
dtype: int64
```

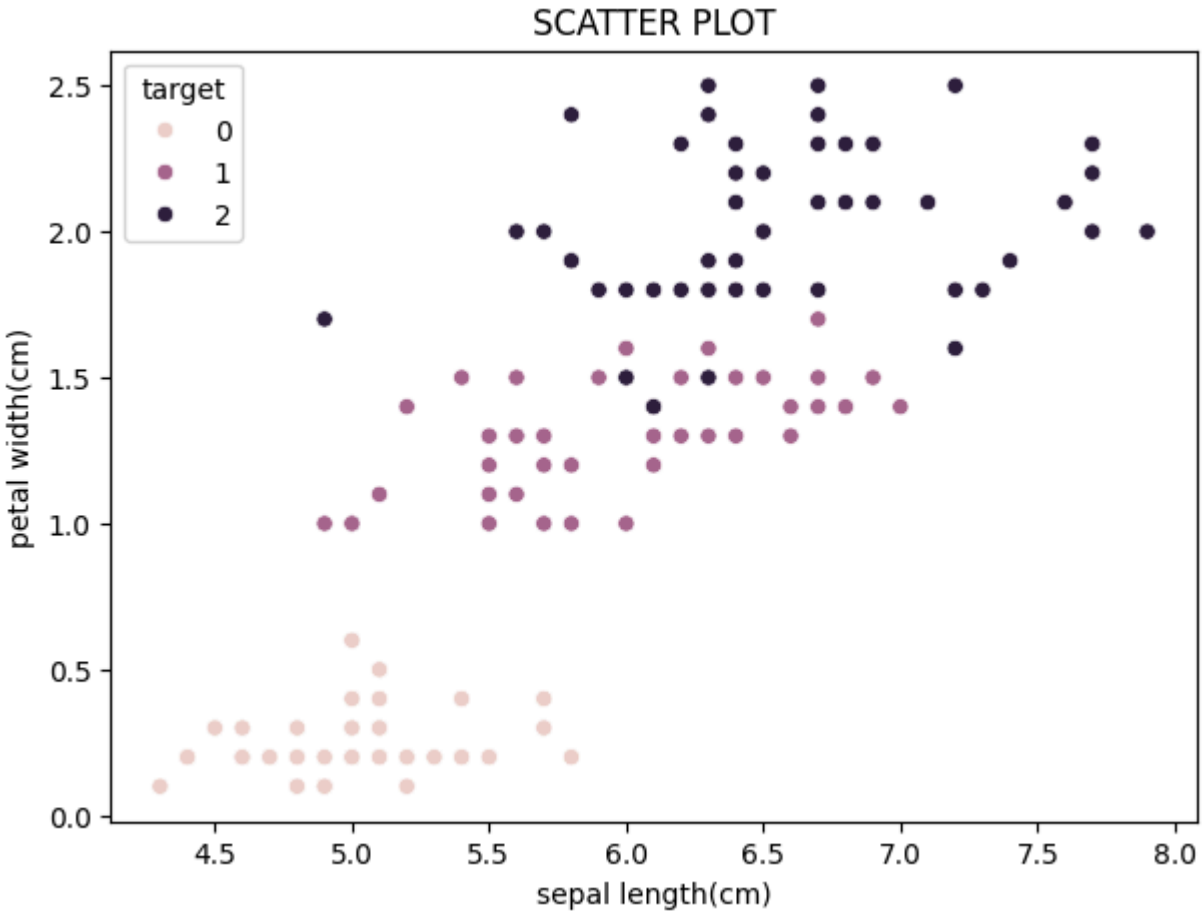
In [36]:
```
#LABEL ENCODING
from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
df["target"]=le.fit_transform(df["target"])
```

In [44]:
```
import matplotlib.pyplot as plt
import seaborn as sns
```

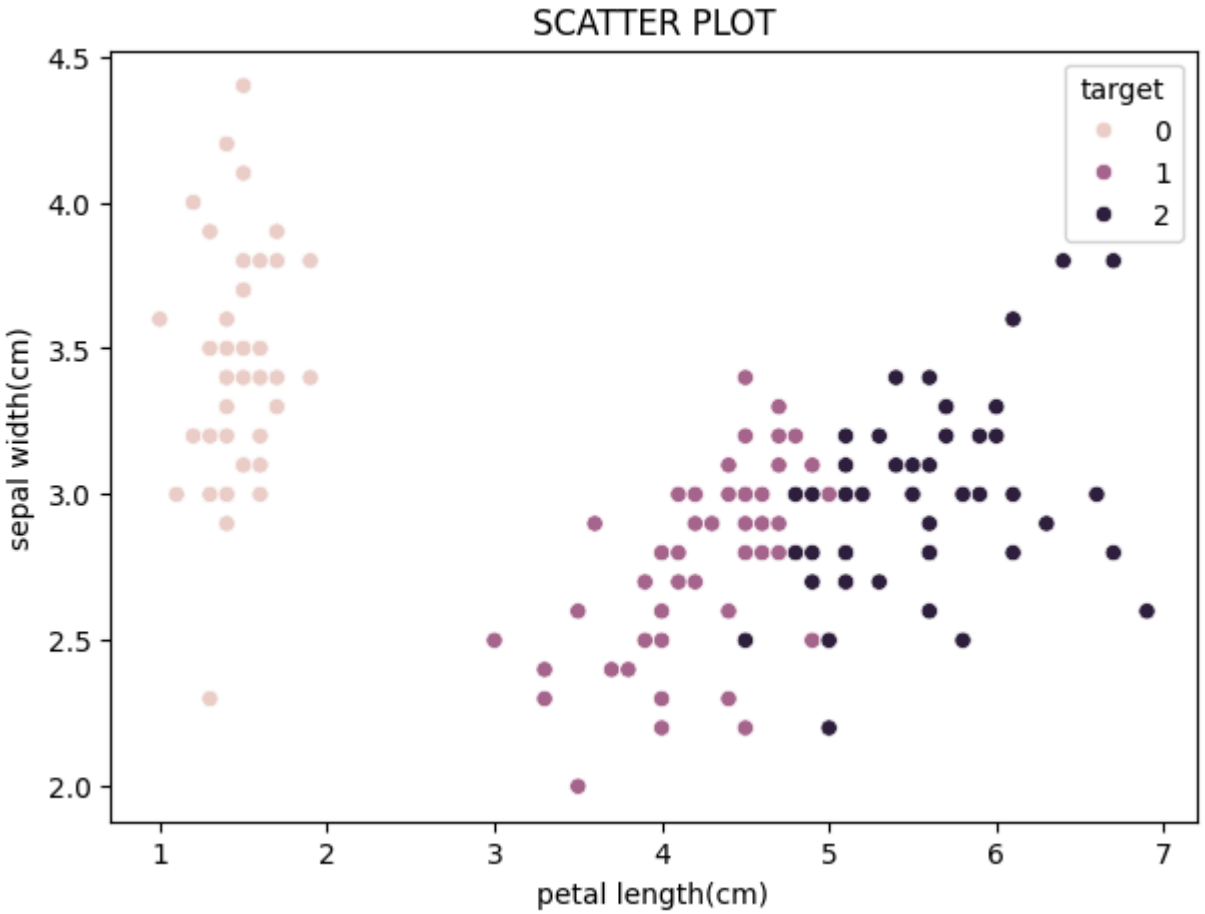
```
plt.figure(figsize=(7,5))
sns.scatterplot(data=df,x="sepal length(cm)",y="sepal width(cm)",hue="target")
plt.title("SCATTER PLOT")
plt.show()
```



In [45]:
```
import matplotlib.pyplot as plt
import seaborn as sns
plt.figure(figsize=(7,5))
sns.scatterplot(data=df,x="sepal length(cm)",y="petal width(cm)",hue="target")
plt.title("SCATTER PLOT")
plt.show()
```



In [46]:
```
import matplotlib.pyplot as plt
import seaborn as sns
plt.figure(figsize=(7,5))
sns.scatterplot(data=df,x="petal length(cm)",y="sepal width(cm)",hue="target")
plt.title("SCATTER PLOT")
plt.show()
```



In [37]:
```
# Separate Features (X) and Target (y)
x=df.drop("target",axis=1)
y=df["target"]
```

In [38]:
```
# Feature Scaling using StandardScaler
from sklearn.preprocessing import StandardScaler
scaler=StandardScaler()
x_scaled=scaler.fit_transform(x)
```

In [41]:
```
# FEATURE SELECTION USING SelectKBest (ANOVA F-test)
from sklearn.feature_selection import SelectKBest,f_classif
```

```
selector=SelectKBest(score_func=f_classif,k=2)
X_selected=selector.fit_transform(x,y)
selected_features=x.columns[selector.get_support()]
selected_features
```

Out[41]:
```
Index(['petal length(cm)', 'petal width(cm)'], dtype='object')
```

In [43]:
```
#SPLITTING DATA INTO TRAIN SET AND TEST SET
from sklearn.model_selection import train_test_split
X_train, X_temp, y_train, y_temp = train_test_split(x, y, test_size=0.2, random_state=42)

X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5, random_state=42)
```

In [ ]:
```
LOAN AMOUNT PREDICTION
```

In [7]:
```
df=pd.read_csv("loan.csv")
print(df.columns)
```

```
Index(['Loan_ID', 'Gender', 'Married', 'Dependents', 'Education',
       'Self_Employed', 'ApplicantIncome', 'CoapplicantIncome', 'LoanAmount',
       'Loan_Amount_Term', 'Credit_History', 'Property_Area', 'Loan_Status'],
      dtype='object')
```

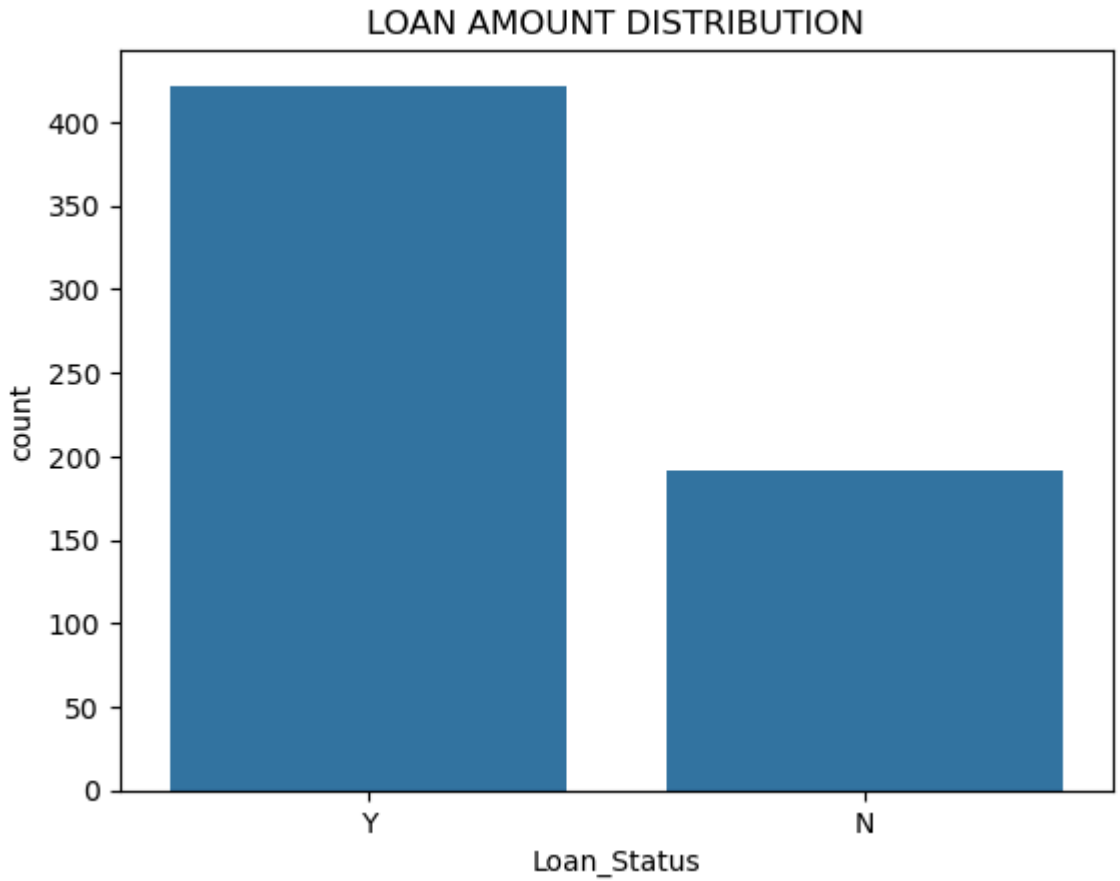
In [8]:
```
df.dtypes
```

Out[8]:
```
Loan_ID               object
Gender                object
Married               object
Dependents            object
Education             object
Self_Employed         object
ApplicantIncome        int64
CoapplicantIncome    float64
LoanAmount           float64
Loan_Amount_Term     float64
Credit_History       float64
Property_Area         object
Loan_Status           object
dtype: object
```

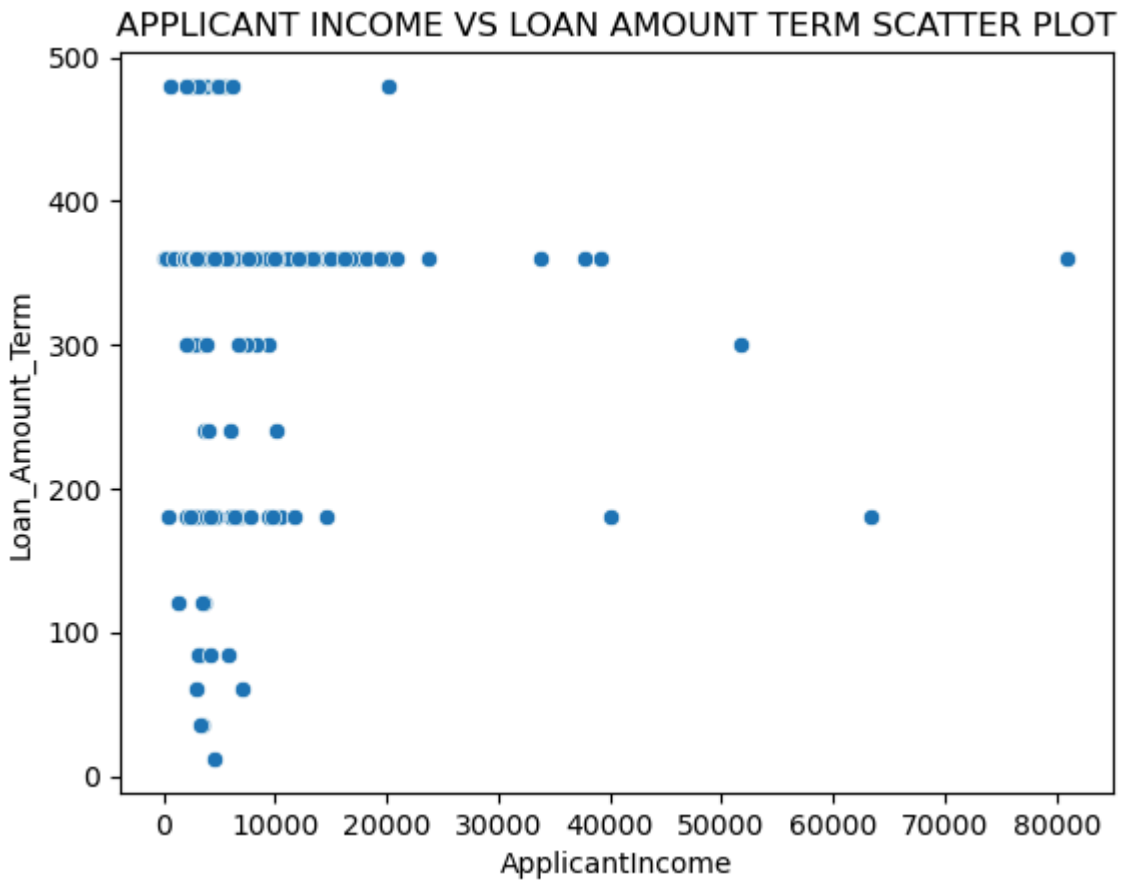
In [9]:
```
df.describe
```

Out[9]:
```
<bound method NDFrame.describe of       Loan_ID  Gender Married Dependents     Education Self_Employed  \
0    LP001002    Male      No          0      Graduate            No
1    LP001003    Male     Yes          1      Graduate            No
2    LP001005    Male     Yes          0      Graduate           Yes
3    LP001006    Male     Yes          0  Not Graduate            No
4    LP001008    Male      No          0      Graduate            No
..        ...     ...     ...        ...           ...           ...
609  LP002978  Female      No          0      Graduate            No
610  LP002979    Male     Yes         3+      Graduate            No
611  LP002983    Male     Yes          1      Graduate            No
612  LP002984    Male     Yes          2      Graduate            No
613  LP002990  Female      No          0      Graduate           Yes

     ApplicantIncome  CoapplicantIncome  LoanAmount  Loan_Amount_Term  \
0               5849                0.0         NaN             360.0
1               4583             1508.0       128.0             360.0
2               3000                0.0        66.0             360.0
3               2583             2358.0       120.0             360.0
4               6000                0.0       141.0             360.0
..               ...                ...         ...               ...
609             2900                0.0        71.0             360.0
610             4106                0.0        40.0             180.0
611             8072              240.0       253.0             360.0
612             7583                0.0       187.0             360.0
613             4583                0.0       133.0             360.0

     Credit_History Property_Area Loan_Status
0               1.0         Urban           Y
1               1.0         Rural           N
2               1.0         Urban           Y
3               1.0         Urban           Y
4               1.0         Urban           Y
..              ...           ...         ...
609             1.0         Rural           Y
610             1.0         Rural           Y
611             1.0         Urban           Y
612             1.0         Urban           Y
613             0.0     Semiurban           N

[614 rows x 13 columns]>
```

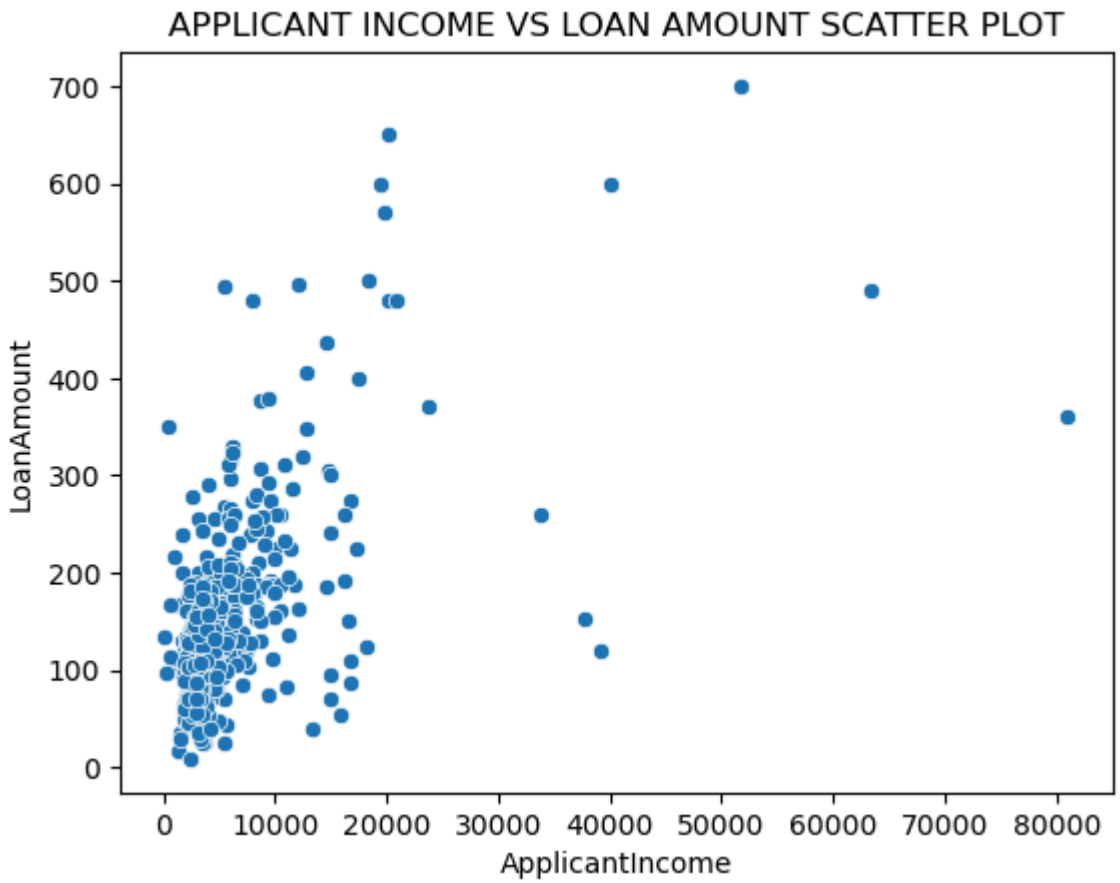
In [10]:
```
import matplotlib.pyplot as plt
import seaborn as sns
sns.countplot(x="Loan_Status",data=df)
plt.title("LOAN AMOUNT DISTRIBUTION")
plt.show()
```



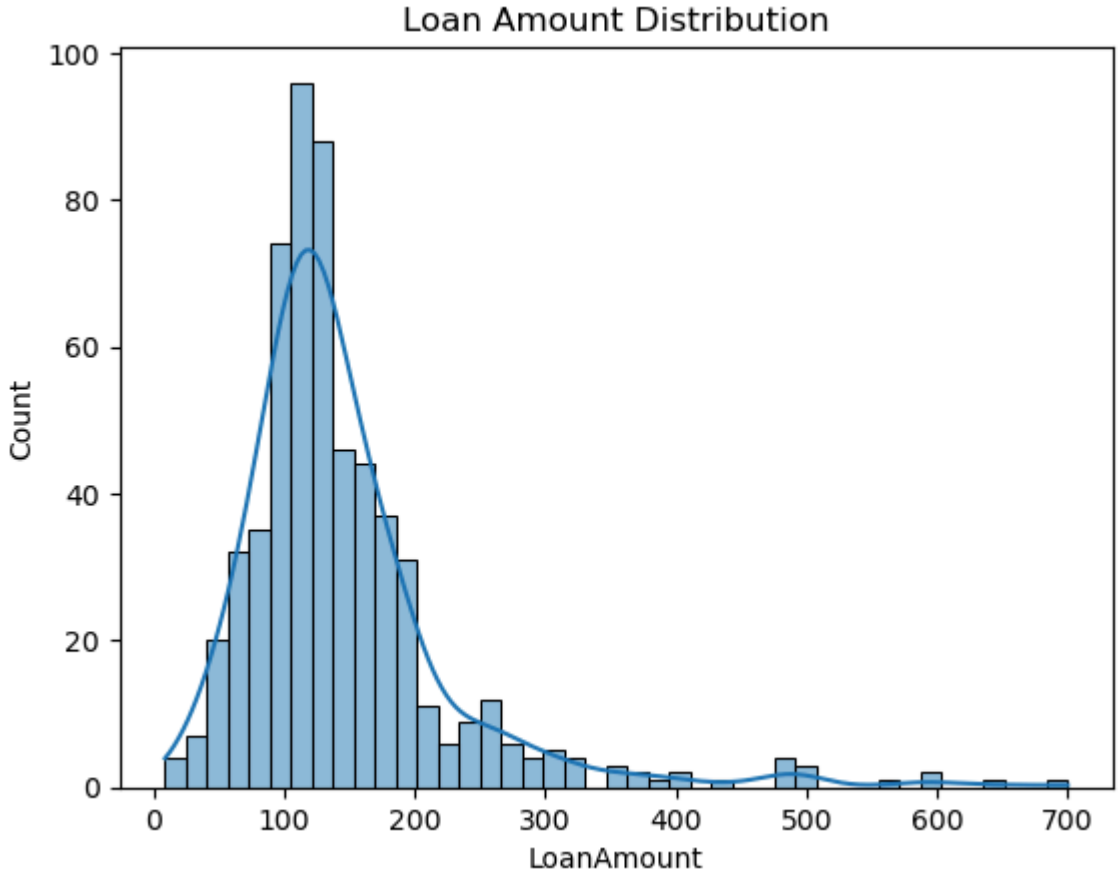
In [15]:
```
sns.scatterplot(x=df["ApplicantIncome"],y=df["Loan_Amount_Term"])
plt.title("APPLICANT INCOME VS LOAN AMOUNT TERM SCATTER PLOT")
plt.show()
```



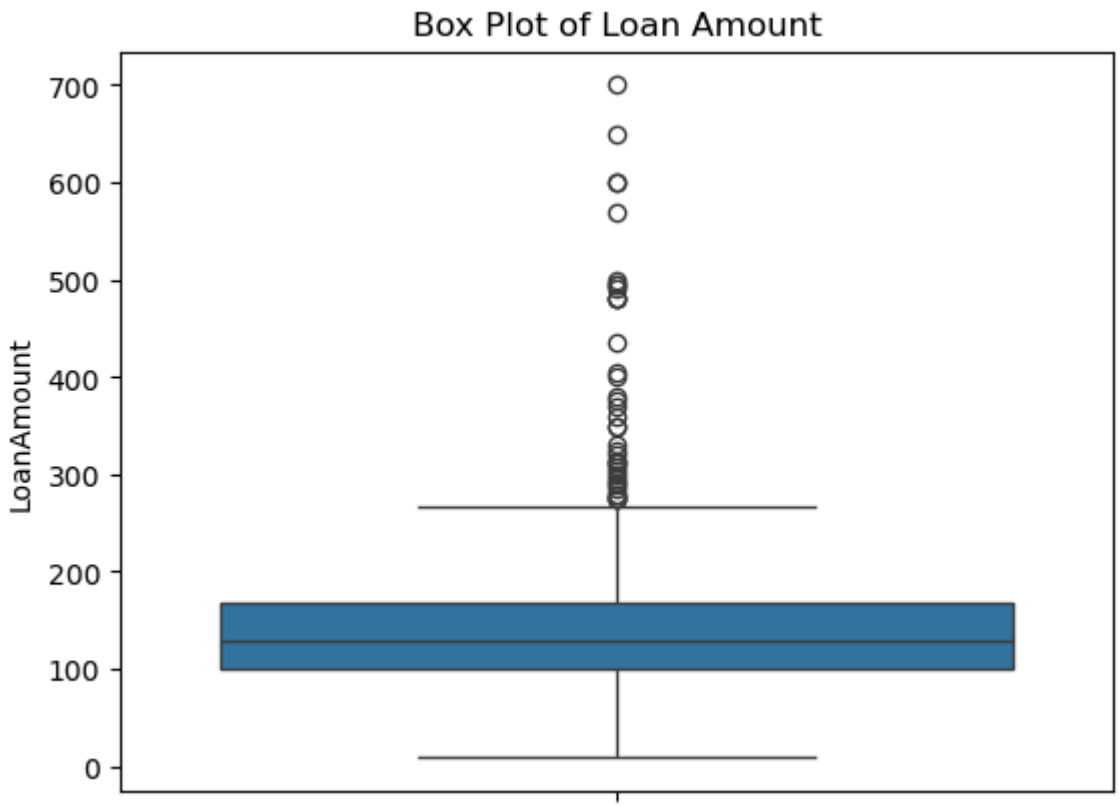
In [16]:
```
sns.scatterplot(x=df["ApplicantIncome"],y=df["LoanAmount"])
plt.title("APPLICANT INCOME VS LOAN AMOUNT SCATTER PLOT")
plt.show()
```



In [17]:
```
sns.histplot(df["LoanAmount"], kde=True)
plt.title("Loan Amount Distribution")
plt.show()
```



In [18]:
```
sns.boxplot(y=df["LoanAmount"])
plt.title("Box Plot of Loan Amount")
plt.show()
```



In [19]:
```
df.isnull().sum()
```

Out[19]:
```
Loan_ID              0
Gender              13
Married              3
Dependents          15
Education            0
Self_Employed       32
ApplicantIncome      0
CoapplicantIncome    0
LoanAmount          22
Loan_Amount_Term    14
Credit_History      50
Property_Area        0
Loan_Status          0
dtype: int64
```

In [23]:
```
df["Gender"].fillna(df["Gender"].mode()[0], inplace=True)
df["Married"].fillna(df["Married"].mode()[0], inplace=True)
df["Dependents"].fillna(df["Dependents"].mode()[0], inplace=True)
df["Self_Employed"].fillna(df["Self_Employed"].mode()[0],inplace=True)
df["LoanAmount"].fillna(df["LoanAmount"].median())
df["Loan_Amount_Term"].fillna(df["Loan_Amount_Term"].mode()[0])
df["Credit_History"].fillna(df["Credit_History"].mode()[0])
```

Out[23]:
```
0      1.0
1      1.0
2      1.0
3      1.0
4      1.0
      ...
609    1.0
610    1.0
611    1.0
612    1.0
613    0.0
Name: Credit_History, Length: 614, dtype: float64
```

In [24]:
```
df.drop("Loan_ID",axis=1,inplace=True)
```

In [25]:
```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
```

```
categorical_cols = df.select_dtypes(include="object").columns
for col in categorical_cols:
    df[col] = le.fit_transform(df[col])
```

In [ ]:
```
DIABETES PREDICTION
```

In [7]:
```
import pandas as pd
df=pd.read_csv("diabetes_prediction_dataset.csv")
print(df.columns)
```

```
Index(['gender', 'age', 'hypertension', 'heart_disease', 'smoking_history',
       'bmi', 'HbA1c_level', 'blood_glucose_level', 'diabetes'],
      dtype='object')
```

In [8]:
```
print(df.describe)
```

```
<bound method NDFrame.describe of        gender   age  hypertension  heart_disease smoking_history    bmi  \
0      Female  80.0             0              1          never  25.19
1      Female  54.0             0              0        No Info  27.32
2        Male  28.0             0              0          never  27.32
3      Female  36.0             0              0        current  23.45
4        Male  76.0             1              1        current  20.14
...       ...   ...           ...            ...            ...    ...
99995  Female  80.0             0              0        No Info  27.32
99996  Female   2.0             0              0        No Info  17.37
99997    Male  66.0             0              0         former  27.83
99998  Female  24.0             0              0          never  35.42
99999  Female  57.0             0              0        current  22.43

       HbA1c_level  blood_glucose_level  diabetes
0              6.6                  140         0
1              6.6                   80         0
2              5.7                  158         0
3              5.0                  155         0
4              4.8                  155         0
...            ...                  ...       ...
99995          6.2                   90         0
99996          6.5                  100         0
99997          5.7                  155         0
99998          4.0                  100         0
99999          6.6                   90         0

[100000 rows x 9 columns]>
```

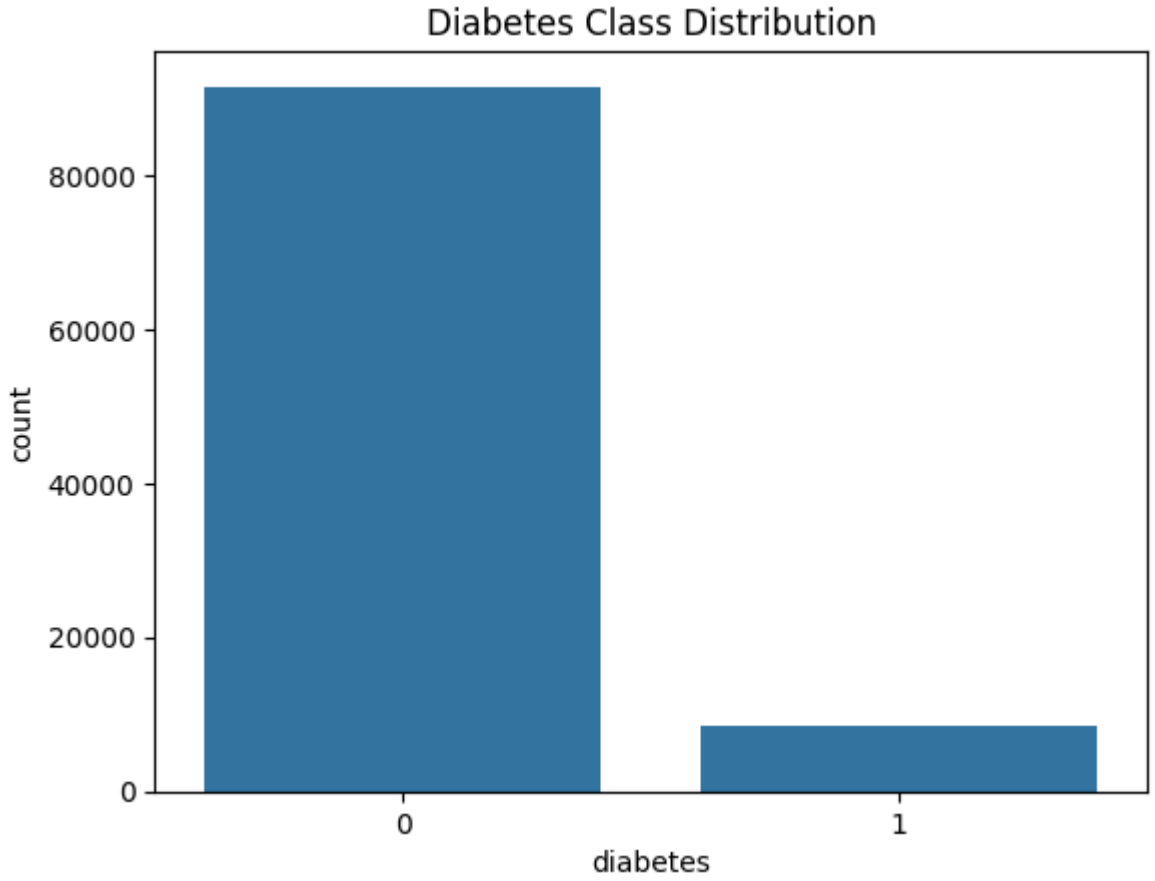
In [9]:
```
print(df.shape)
```

```
(100000, 9)
```

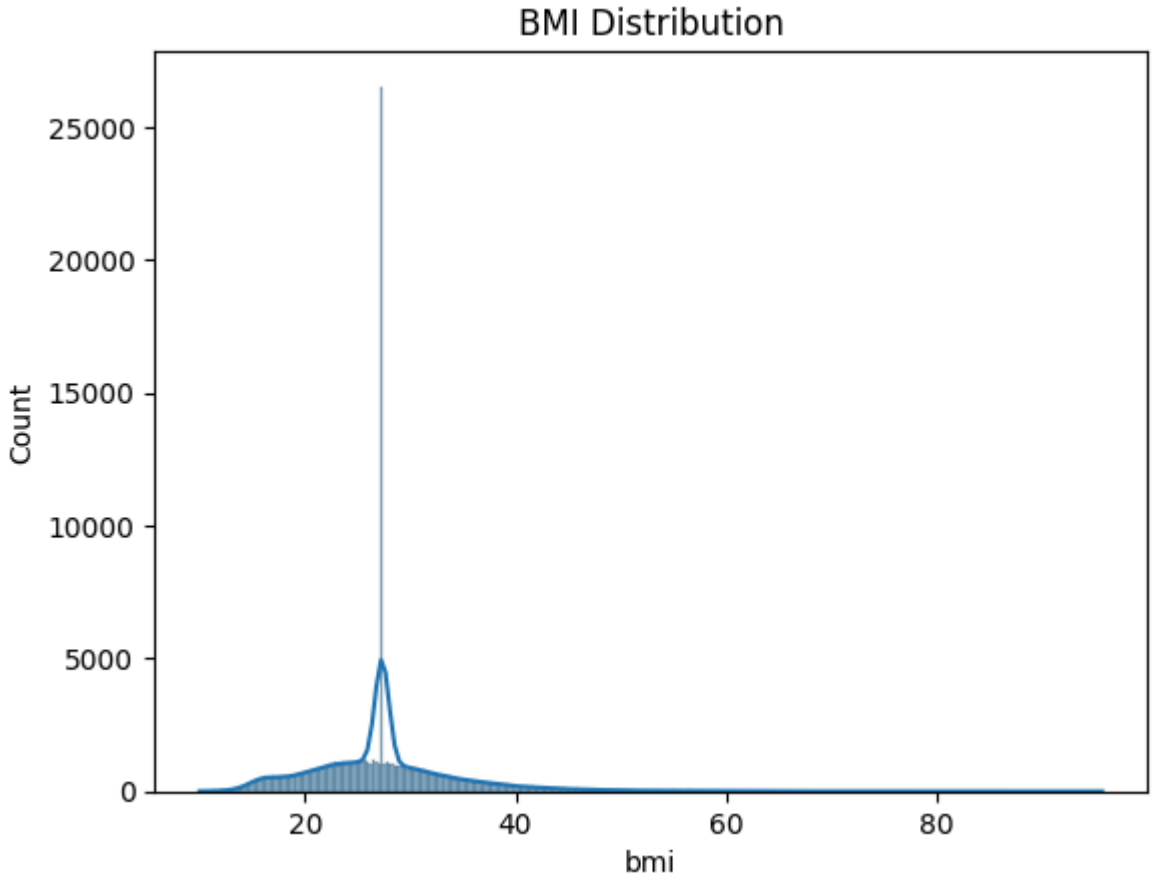
In [10]:
```
df.isnull().sum()
```

Out[10]:
```
gender                0
age                   0
hypertension          0
heart_disease         0
smoking_history       0
bmi                   0
HbA1c_level           0
blood_glucose_level   0
diabetes              0
dtype: int64
```

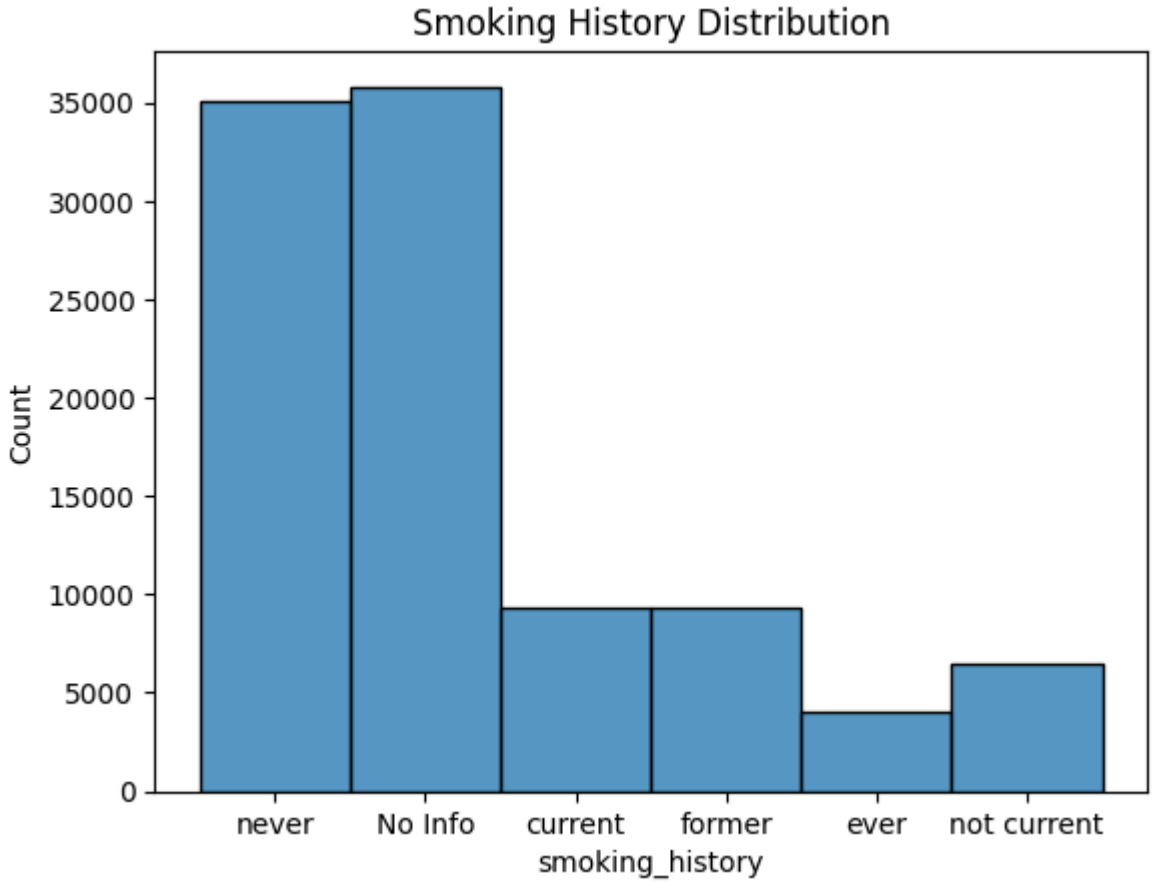
In [11]:
```
import matplotlib.pyplot as plt
import seaborn as sns
sns.countplot(x="diabetes",data=df)
plt.title("Diabetes Class Distribution")
plt.show()
```



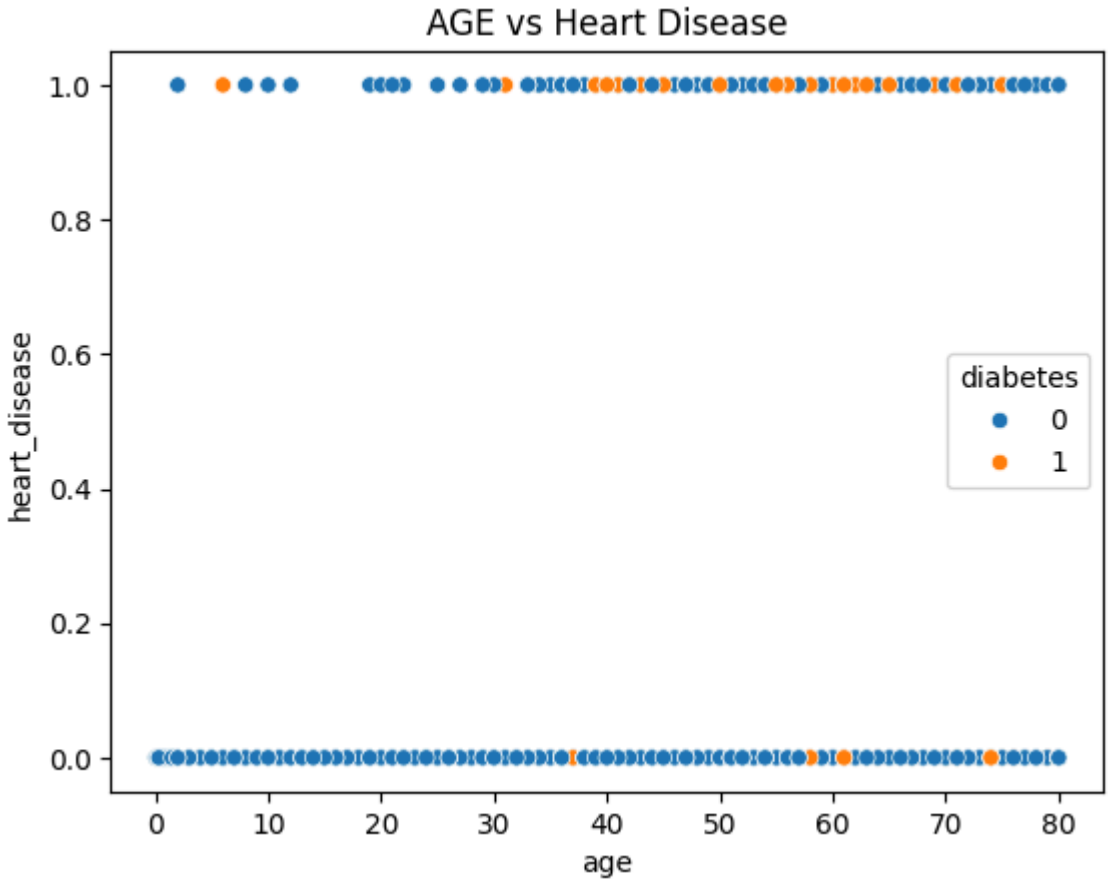
In [13]:
```
sns.histplot(df["bmi"],kde="true")
plt.title("BMI Distribution")
plt.show()
```



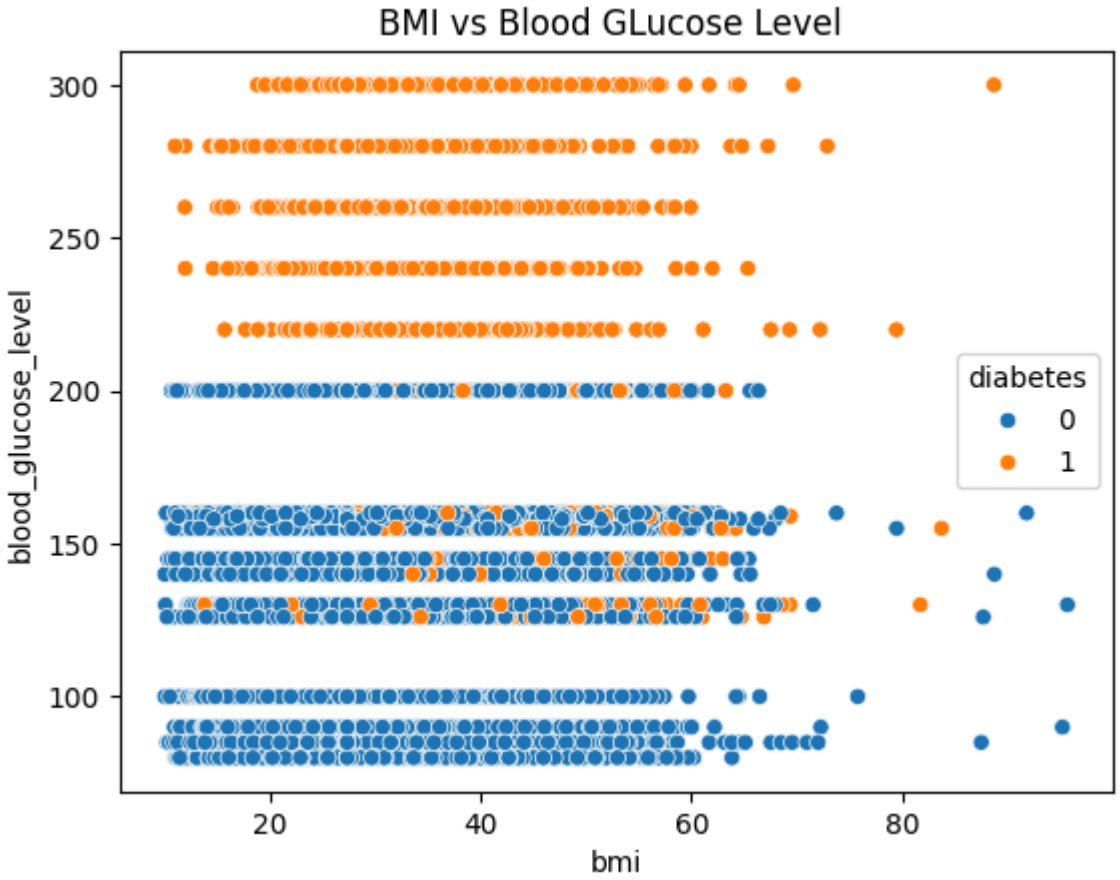
In [16]:
```
sns.histplot(df["smoking_history"])
plt.title("Smoking History Distribution")
plt.show()
```



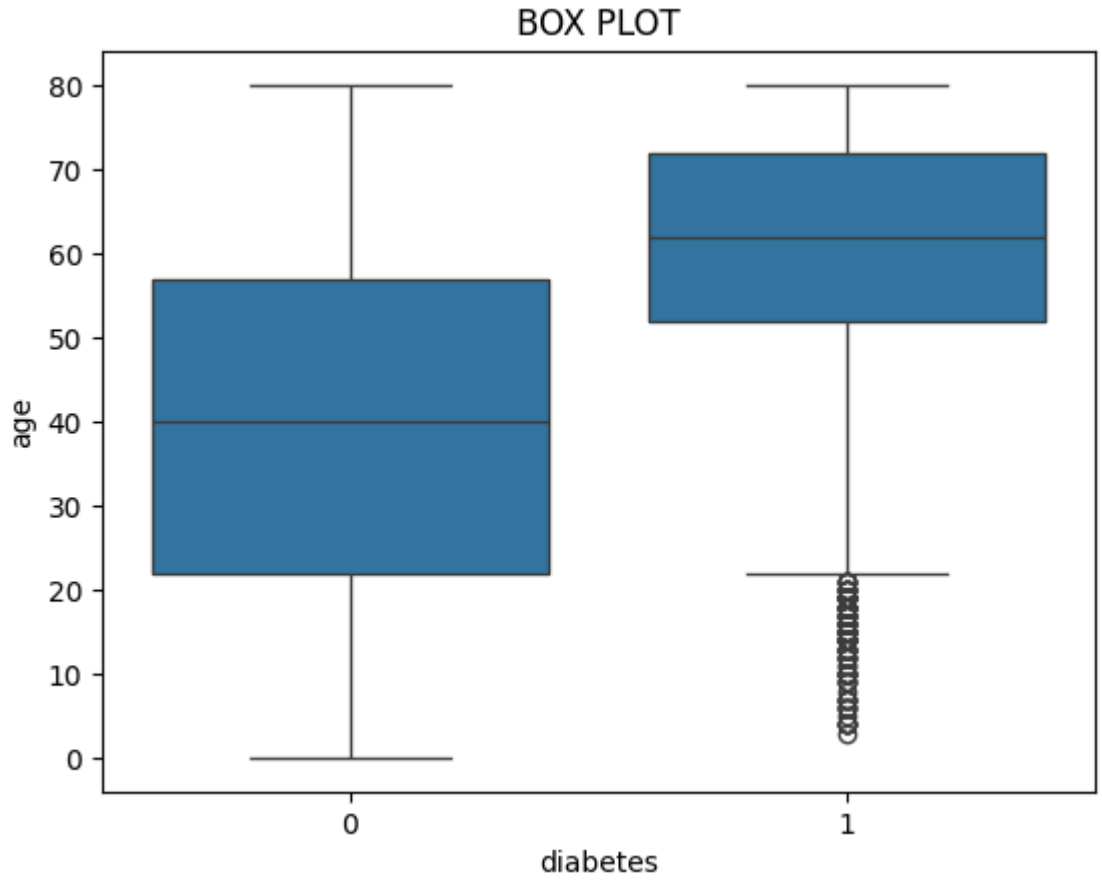
In [18]:
```
sns.scatterplot(data=df,x="age",y="heart_disease",hue="diabetes")
plt.title("AGE vs Heart Disease")
plt.show()
```



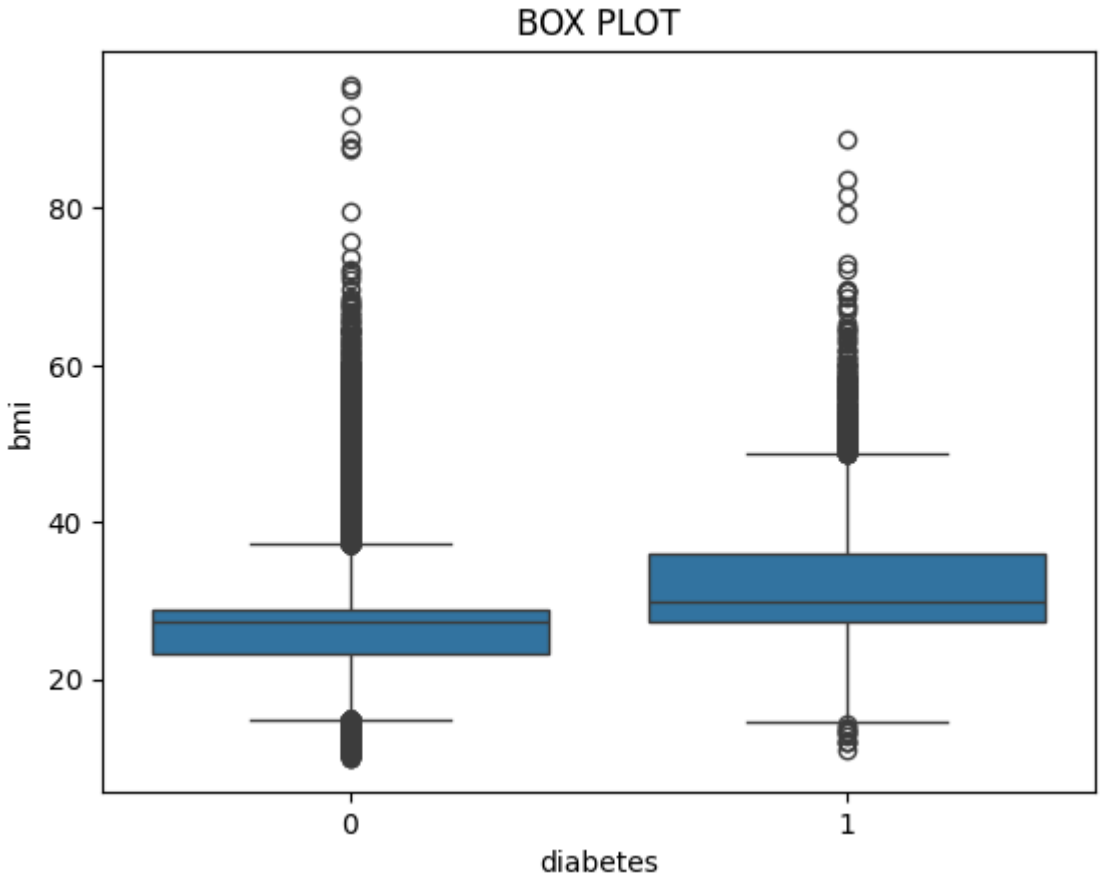
In [19]:
```
sns.scatterplot(data=df,x="bmi",y="blood_glucose_level",hue="diabetes")
plt.title("BMI vs Blood GLucose Level ")
plt.show()
```



In [20]:
```
sns.boxplot(x="diabetes",y="age",data=df)
plt.title("BOX PLOT")
plt.show()
```



In [22]:
```
sns.boxplot(x="diabetes",y="bmi",data=df)
plt.title("BOX PLOT")
plt.show()
```



In [31]:
```
from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
df["diabetes"]=le.fit_transform(df["diabetes"])
df["gender"]=le.fit_transform(df["gender"])
df["smoking_history"]=le.fit_transform(df["smoking_history"])
```

In [28]:
```
df.dtypes
```

Out[28]:
```
gender                object
age                  float64
hypertension           int64
heart_disease          int64
smoking_history       object
bmi                  float64
HbA1c_level          float64
blood_glucose_level    int64
diabetes               int64
dtype: object
```

In [32]:
```
x=df.drop("diabetes",axis=1)
y=df["diabetes"]

from sklearn.preprocessing import StandardScaler
scaled=StandardScaler()
x_scaled=scaled.fit_transform(x)
```

In [34]:
```
from sklearn.feature_selection import SelectKBest,f_classif
selector=SelectKBest(score_func=f_classif,k=5)
x_selected=selector.fit_transform(x,y)
selected_features=x.columns[selector.get_support()]
selected_features
```

Out[34]:
```
Index(['age', 'hypertension', 'bmi', 'HbA1c_level', 'blood_glucose_level'], dtype='object')
```

In [ ]:
```
EMAIL CLASSIFICATION
```

In [35]:
```
df=pd.read_csv("emails.csv")
```

In [36]:
```
print(df.columns)
```

```
Index(['Email No.', 'the', 'to', 'ect', 'and', 'for', 'of', 'a', 'you', 'hou',
       ...
       'connevey', 'jay', 'valued', 'lay', 'infrastructure', 'military',
       'allowing', 'ff', 'dry', 'Prediction'],
      dtype='object', length=3002)
```

In [37]:
```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5172 entries, 0 to 5171
Columns: 3002 entries, Email No. to Prediction
dtypes: int64(3001), object(1)
memory usage: 118.5+ MB
None
```

In [38]:
```
print(df.shape)
```

```
(5172, 3002)
```

In [40]:
```
df.describe()
```

Out[40]:

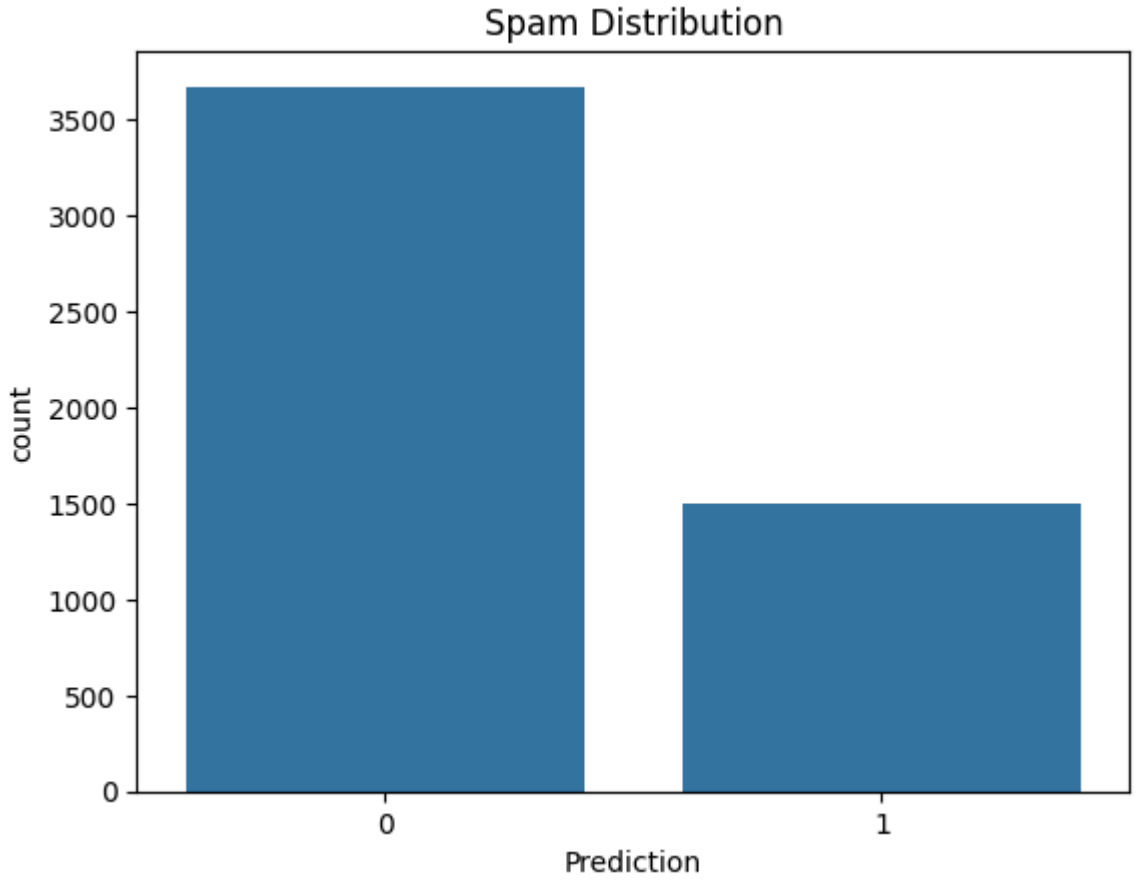
	the	to	ect	and	for	of	a	you	hou
count	5172.000000	5172.000000	5172.000000	5172.000000	5172.000000	5172.000000	5172.000000	5172.000000	5172.000000
mean	6.640565	6.188128	5.143852	3.075599	3.124710	2.627030	55.517401	2.466551	2.024362
std	11.745009	9.534576	14.101142	6.045970	4.680522	6.229845	87.574172	4.314444	6.967878
min	0.000000	0.000000	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.000000	1.000000	1.000000	0.000000	1.000000	0.000000	12.000000	0.000000	0.000000
50%	3.000000	3.000000	1.000000	1.000000	2.000000	1.000000	28.000000	1.000000	0.000000
75%	8.000000	7.000000	4.000000	3.000000	4.000000	2.000000	62.250000	3.000000	1.000000
max	210.000000	132.000000	344.000000	89.000000	47.000000	77.000000	1898.000000	70.000000	167.000000

8 rows × 3001 columns

In [39]:
```
df.isnull().sum()
```

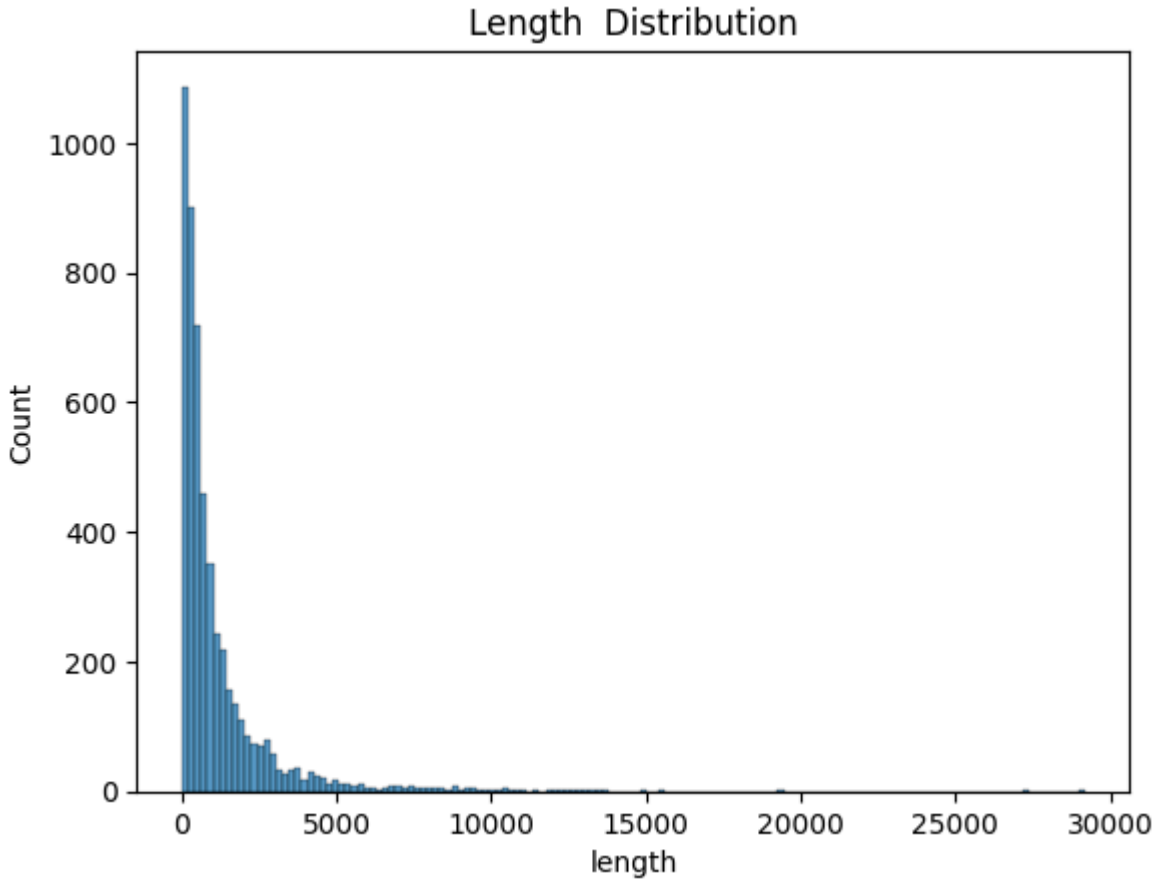
Out[39]:
```
Email No.     0
the           0
to            0
ect           0
and           0
              ..
military      0
allowing      0
ff            0
dry           0
Prediction    0
Length: 3002, dtype: int64
```

In [41]:
```
sns.countplot(x="Prediction",data=df)
plt.title("Spam Distribution")
plt.show()
```

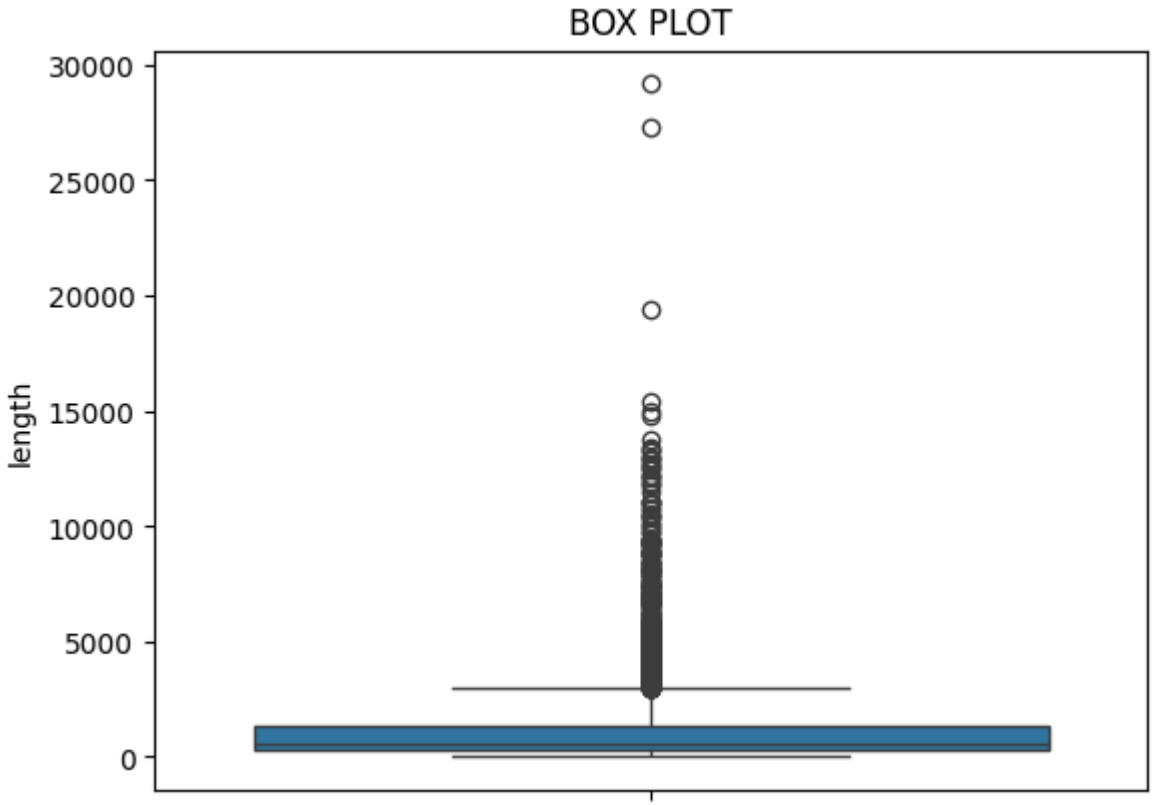


In [46]:
```
num=df.select_dtypes(include="number")
df["length"]=num.sum(axis=1)
```

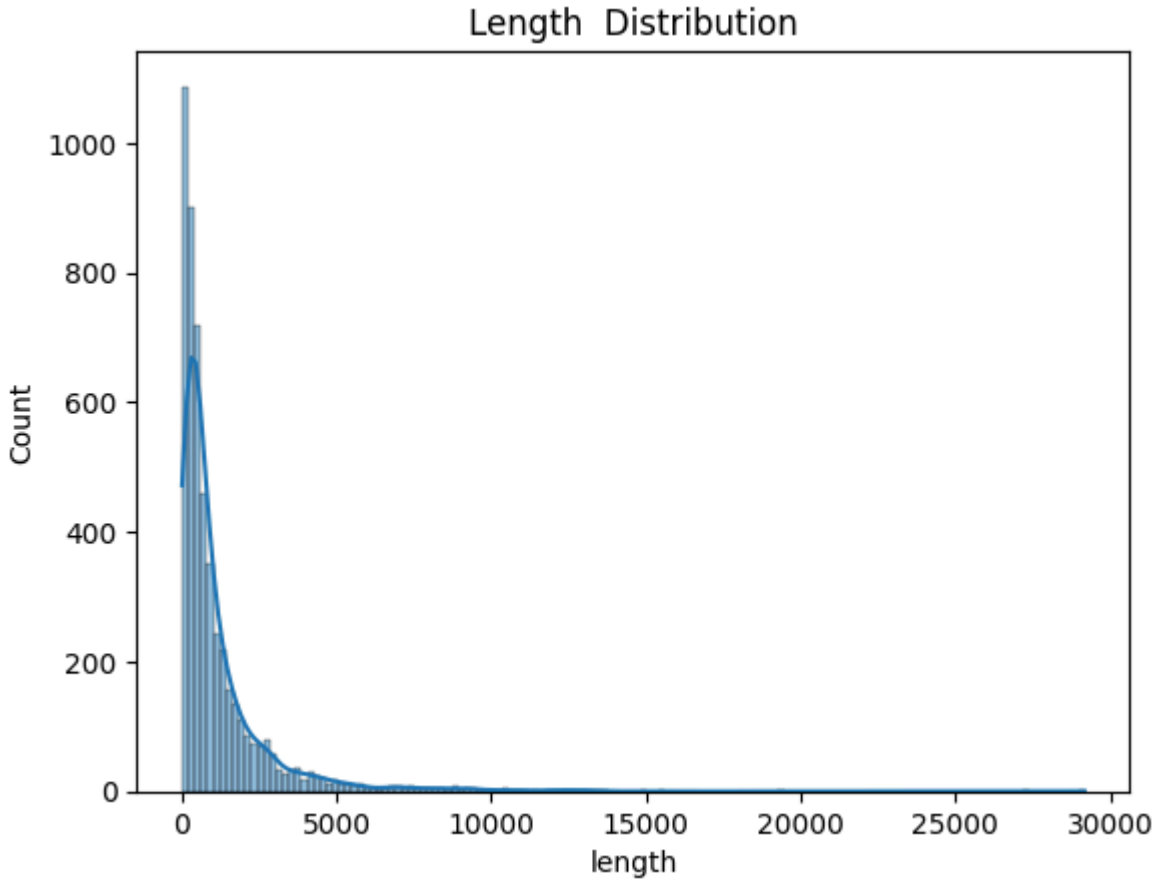
In [47]:
```
sns.histplot(df["length"])
plt.title("Length  Distribution")
plt.show()
```



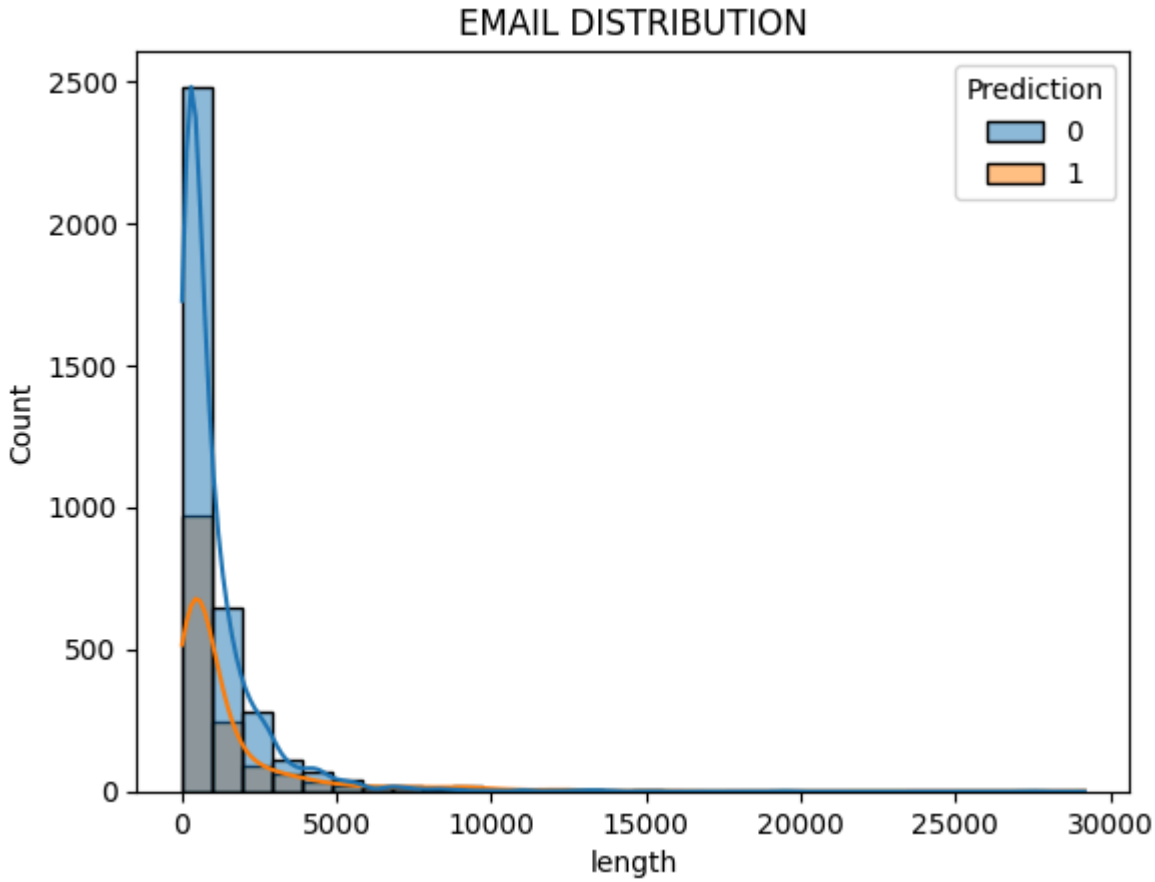
In [51]:
```
sns.boxplot(y=df["length"])
plt.title("BOX PLOT")
plt.show()
```



In [49]:
```
sns.histplot(df["length"],kde="true")
plt.title("Length  Distribution")
plt.show()
```



In [52]:
```
sns.histplot(data=df,x="length",hue="Prediction",kde="true",bins=30)
plt.title("EMAIL DISTRIBUTION")
plt.show()
```



In [55]:
```
df.dtypes
```

Out[55]:
```
Email No.     object
the            int64
to             int64
ect            int64
and            int64
               ...
military       int64
allowing       int64
ff             int64
dry            int64
Prediction     int64
Length: 3002, dtype: object
```

In [57]:
```
#LABEL ENCODING
from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
df["Email No."]=le.fit_transform(df["Email No."])
```

In [58]:
```
# Separate Features (X) and Target (y)
x=df.drop("Prediction",axis=1)
y=df["Prediction"]

# Feature Scaling using StandardScaler
from sklearn.preprocessing import StandardScaler
scaled=StandardScaler()
x_scaled=scaled.fit_transform(x)
```

In [59]:
```
# FEATURE SELECTION USING SelectKBest (ANOVA F-test)
from sklearn.feature_selection import SelectKBest,f_classif
selector=SelectKBest(score_func=f_classif,k=5)
x_selected=selector.fit_transform(x,y)
selected_features=x.columns[selector.get_support()]
selected_features
```

Out[59]:
```
Index(['hpl', 'more', 'thanks', 'hanks', 'thank'], dtype='object')
```

In [ ]:
```
DIGIT RECOGNIZER
```

In [3]:
```
import pandas as pd
#LOADING THE DATASET
df=pd.read_csv("mnist_train.csv")
```

In [4]:
```
#STATISTICS
df.describe()
```

Out[4]:

	5	0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	...	0.608	
count	59999.000000	59999.0	59999.0	59999.0	59999.0	59999.0	59999.0	59999.0	59999.0	59999.0	...	59999.000000	5999
mean	4.453924	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.200437	
std	2.889294	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	6.042522	
min	0.000000	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.000000	
25%	2.000000	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.000000	
50%	4.000000	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.000000	
75%	7.000000	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.000000	
max	9.000000	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	254.000000	25

8 rows × 785 columns

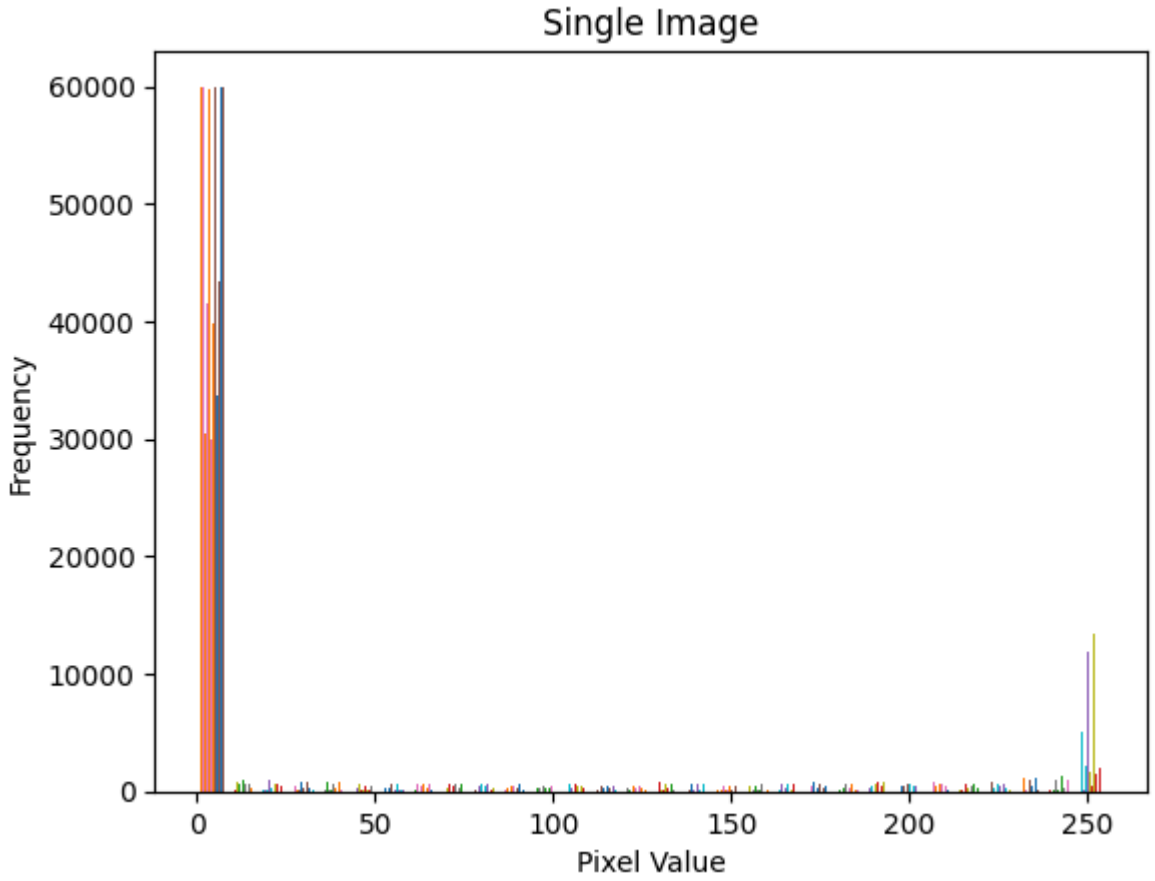
In [63]:
```
#NUMBER OF FEATURES
print(df.columns)
```

```
Index(['5', '0', '0.1', '0.2', '0.3', '0.4', '0.5', '0.6', '0.7', '0.8',
       ...
       '0.608', '0.609', '0.610', '0.611', '0.612', '0.613', '0.614', '0.615',
       '0.616', '0.617'],
      dtype='object', length=785)
```

In [65]:
```
#NO OF ROWS AND COLUMNS
print(df.shape)
```

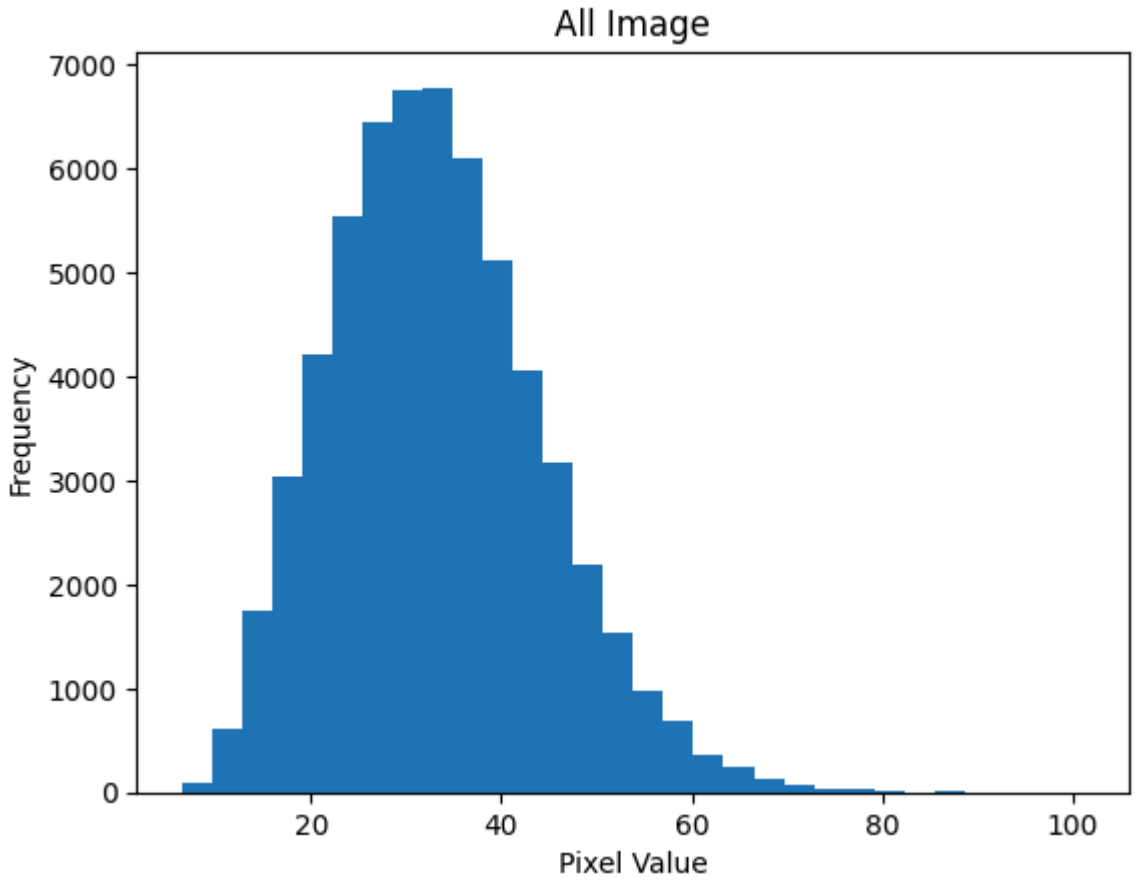
```
(59999, 785)
```

In [73]:
```
#HISTOGRAM PLOT(SINGLE IMAGE)
plt.hist(df.iloc[0:-1],bins=30)
plt.title("Single Image")
plt.xlabel("Pixel Value")
plt.ylabel("Frequency")
plt.show()
```

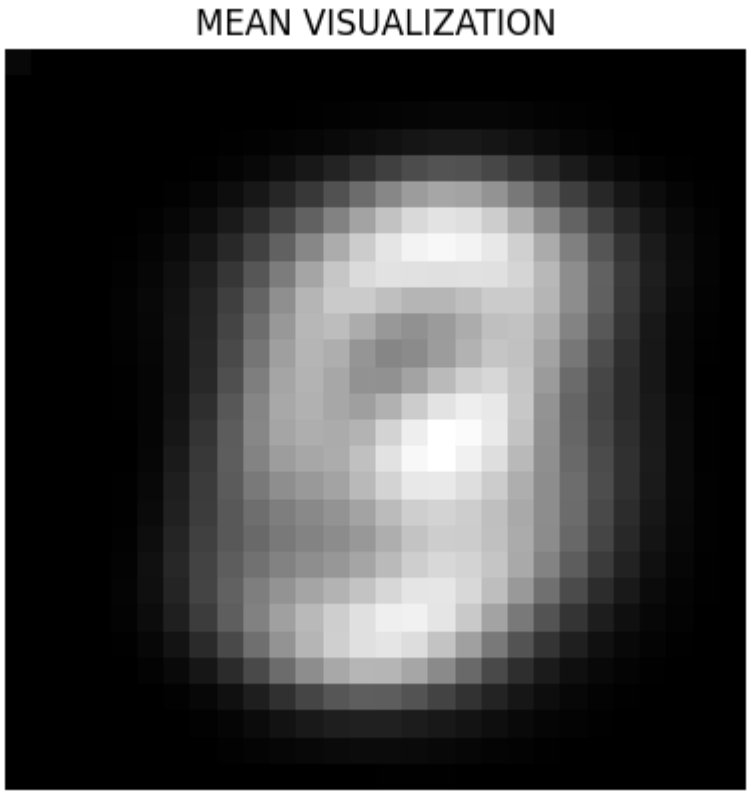


In [70]:
```
#HISTOGRAM PLOT(ALL IMAGES)
avg=df.mean(axis=1)
plt.hist(avg,bins=30)
plt.title("All Image")
plt.xlabel("Pixel Value")
```

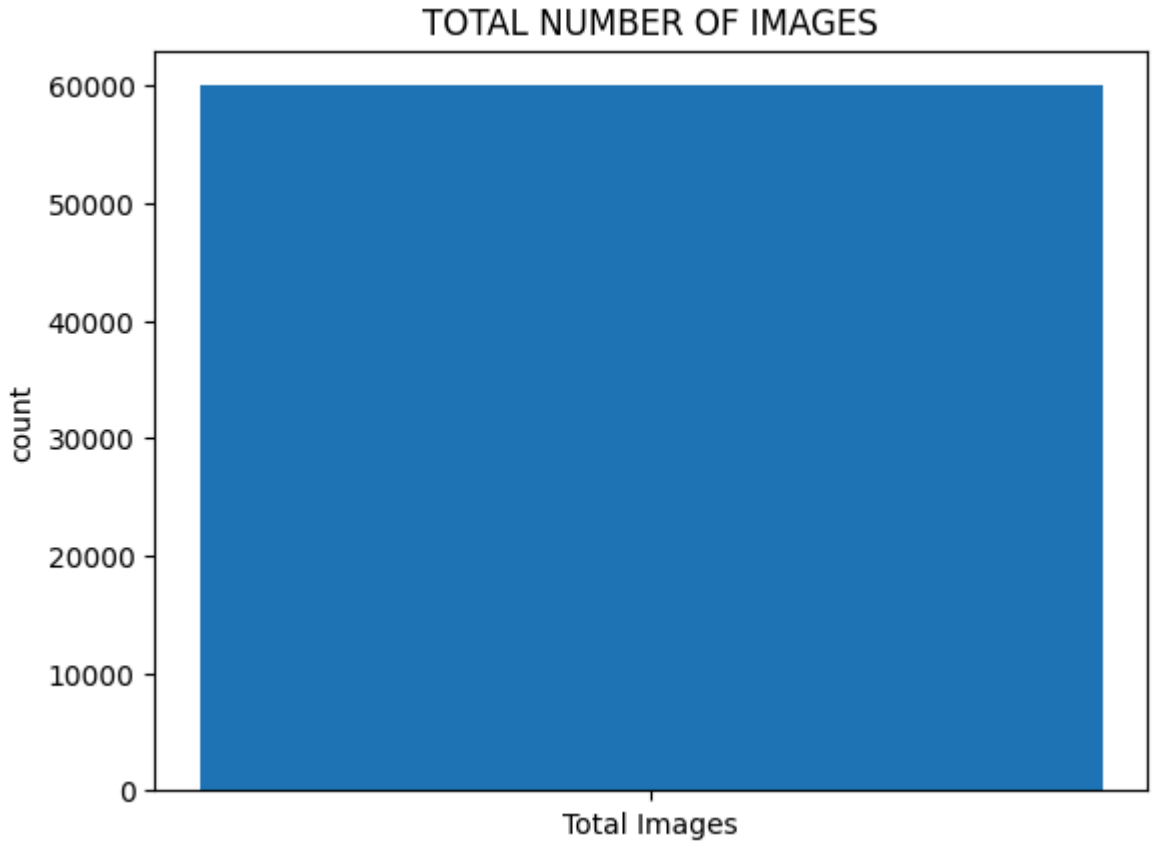
```
plt.ylabel("Frequency")
plt.show()
```



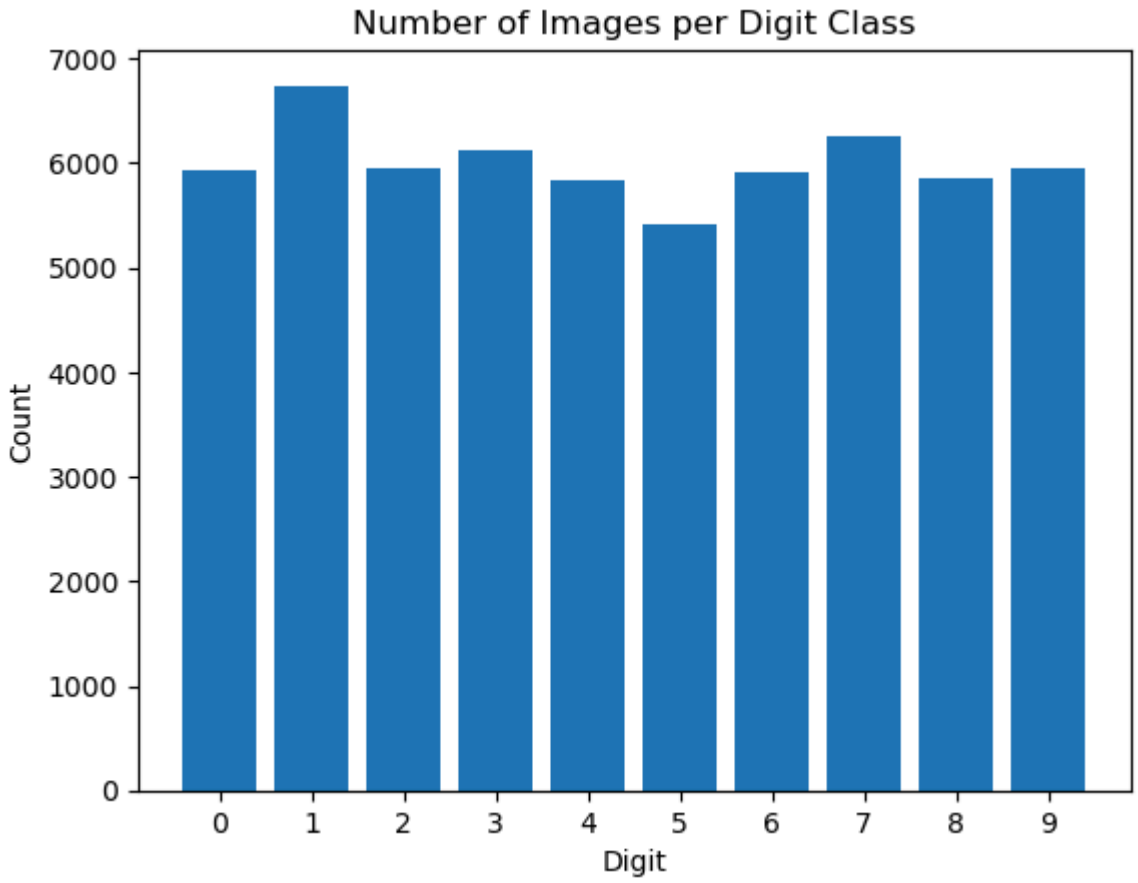
In [77]:
```
#MEAN VISUALIZATION
import numpy as np
m=df.iloc[:,:-1].mean(axis=0).values
image=m.reshape(28,28)
plt.imshow(image,cmap="gray")
plt.title("MEAN VISUALIZATION")
plt.axis("off")
plt.show()
```



In [78]:
```
#TO FIND THE TOTAL NUMBER OF IMAGES
total=len(df)
plt.bar(["Total Images"],[total])
plt.title("TOTAL NUMBER OF IMAGES")
plt.ylabel("count")
plt.show()
```



In [6]:
```
# TO FIND THE NUMBER OF IMAGES PER CLASS
import numpy as np
import matplotlib.pyplot as plt
labels = df.iloc[:, 0]
digits, counts = np.unique(labels, return_counts=True)
plt.bar(digits, counts)
plt.title("Number of Images per Digit Class")
plt.xlabel("Digit")
plt.ylabel("Count")
plt.xticks(digits)
plt.show()
```


Number of Images per Digit Class
Count
Digit
0
1000
2000
3000
4000
5000
6000
7000
0
1
2
3
4
5
6
7
8
9